```python
"""

Generates and saves weather charts using matplotlib.

Charts produced:
  1. Temperature trend line chart (7-day forecast)
  2. Wind speed bar chart (7-day forecast)
  3. Combined dashboard chart (temperature + wind + condition labels)
"""

import os
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from datetime import date

from models import WeatherRecord


# Output folder

CHARTS_DIR = os.path.join(os.path.dirname(__file__), "data", "charts")


def _ensure_charts_dir() -> None:
    """Create the charts/ directory if it does not exist."""
    os.makedirs(CHARTS_DIR, exist_ok=True)


# Color helper map weather description to a bar color


# Dictionary: weather condition - hex color Data types topic
_CONDITION_COLORS: dict[str, str] = {
    "Clear sky":        "#FFD700",
    "Mainly clear":     "#FFE066",
    "Partly cloudy":    "#B0C4DE",
    "Overcast":         "#808080",
```

```
    "Fog":                "#A9A9A9",
    "Light drizzle":      "#87CEEB",
    "Moderate drizzle": "#6495ED",
    "Dense drizzle":      "#4169E1",
    "Slight rain":        "#6495ED",
    "Moderate rain":      "#4169E1",
    "Heavy rain":         "#00008B",
    "Slight snow":        "#ADD8E6",
    "Moderate snow":      "#87CEFA",
    "Heavy snow":         "#E0FFFF",
    "Slight showers":     "#87CEEB",
    "Thunderstorm":       "#4B0082",
}

DEFAULT_COLOR = "#90EE90"


def _condition_color(description: str) -> str:
    """Return a hex color for the given weather condition."""
    return _CONDITION_COLORS.get(description, DEFAULT_COLOR)


# Chart 1  Temperature line chart

def plot_temperature_trend(
    forecast: list[WeatherRecord],
    city_name: str,
) -> str:
    """
    Create a 7-day temperature line chart and save it as a PNG.

    Returns:
        str: file path of the saved chart.
    """
    _ensure_charts_dir()


    dates: list[str] = [str(r.recorded_date)[5:] for
```

```
r in forecast]   # MM-DD
    temps: list[float] = [r.temperature_c for r in forecast]

    fig, ax = plt.subplots(figsize=(9, 4))


    ax.plot(dates, temps, color="#E05C2A", linewidth=2.5,
            marker="o", markersize=7, markerfacecolor="white",
            markeredgecolor="#E05C2A", markeredgewidth=2, zorder=3)

    ax.fill_between(dates, temps, min(temps) - 1,
                    alpha=0.15, color="#E05C2A")


    for x, y in zip(dates, temps):
        ax.annotate(
            f"{y}°C",
            xy=(x, y),
            xytext=(0, 10),
            textcoords="offset points",
            ha="center",
            fontsize=9,
            color="#333333",
        )


    ax.set_title(f"7-Day Temperature Trend — {city_name}",
                 fontsize=13, fontweight="bold", pad=14)
    ax.set_xlabel("Date", fontsize=10)
    ax.set_ylabel("Temperature (°C)", fontsize=10)
    ax.grid(axis="y", linestyle="--", alpha=0.4)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.tick_params(axis="x", rotation=15)

```

```python
    fig.tight_layout()


    filename = os.path.join(CHARTS_DIR, f"{city_name.lower()}_temperature.png")
    fig.savefig(filename, dpi=130, bbox_inches="tight")
    plt.close(fig)

    return filename


# Chart 2 — Wind speed bar chart

def plot_wind_speed(
    forecast: list[WeatherRecord],
    city_name: str,
) -> str:
    """
    Create a 7-day wind speed bar chart and save it as a PNG.

    Returns:
        str: file path of the saved chart.
    """
    _ensure_charts_dir()

    dates: list[str] = [str(r.recorded_date)[5:] for r in forecast]
    winds: list[float] = [r.wind_speed_kmh for r in forecast]


    bar_colors: list[str] = [
        "#E05C2A" if w >= 30 else "#F4A460" if w >= 15 else "#87CEEB"
        for w in winds
    ]

    fig, ax = plt.subplots(figsize=(9, 4))

```

```
    bars = ax.bar(dates, winds, color=bar_colors, edgecolor="white",
                  linewidth=0.8, width=0.6)


    for bar, val in zip(bars, winds):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.5,
            f"{val}",
            ha="center", va="bottom",
            fontsize=9, color="#333333",
        )


    legend_items = [
        mpatches.Patch(color="#87CEEB", label="Light (< 15 km/h)"),
        mpatches.Patch(color="#F4A460", label="Moderate (15-30 km/h)"),
        mpatches.Patch(color="#E05C2A", label="Strong (> 30 km/h)"),
    ]
    ax.legend(handles=legend_items, fontsize=8, loc="upper right")

    ax.set_title(f"7-Day Wind Speed — {city_name}",
                 fontsize=13, fontweight="bold", pad=14)
    ax.set_xlabel("Date", fontsize=10)
    ax.set_ylabel("Wind Speed (km/h)", fontsize=10)
    ax.grid(axis="y", linestyle="--", alpha=0.4)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.tick_params(axis="x", rotation=15)

    fig.tight_layout()

    filename = os.path.join(CHARTS_DIR, f"{city_name.lower()}_wind.png")
    fig.savefig(filename, dpi=130, bbox_inches="
```

```
tight")
    plt.close(fig)

    return filename



# Chart 3  Combined dashboard temperature  wind side by side


def plot_dashboard(
    forecast: list[WeatherRecord],
    current: WeatherRecord,
) -> str:
    """
    Create a combined 2-panel dashboard chart and save it as a PNG.
    Top panel: temperature line.
    Bottom panel: wind bar chart.
    Includes condition labels below date axis.

    Returns:
        str: file path of the saved chart.
    """
    _ensure_charts_dir()

    city_name: str = current.city


    dates: list[str] = [str(r.recorded_date)[5:] for r in forecast]
    temps: list[float] = [r.temperature_c for r in forecast]
    winds: list[float] = [r.wind_speed_kmh for r in forecast]
    conditions: list[str] = [r.weather_description for r in forecast]
    colors: list[str] = [_condition_color(c) for c in conditions]

```

```python
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
    fig.suptitle(
        f"Weather Dashboard — {city_name}  |  "
        f"Today: {current.temperature_c}°C, {current.weather_description}",
        fontsize=13, fontweight="bold", y=0.98,
    )


    ax1.plot(dates, temps, color="#E05C2A", linewidth=2.5,
             marker="o", markersize=7,
             markerfacecolor="white", markeredgecolor="#E05C2A",
             markeredgewidth=2, zorder=3)
    ax1.fill_between(dates, temps, min(temps) - 1,
                     alpha=0.12, color="#E05C2A")

    for x, y in zip(dates, temps):
        ax1.annotate(f"{y}°C", xy=(x, y), xytext=(0, 9),
                     textcoords="offset points",
                     ha="center", fontsize=8.5, color="#444")

    ax1.set_ylabel("Temperature (°C)", fontsize=10)
    ax1.grid(axis="y", linestyle="--", alpha=0.35)
    ax1.spines["top"].set_visible(False)
    ax1.spines["right"].set_visible(False)
    ax1.set_title("Temperature Trend (°C)", fontsize=10,
                  loc="left", color="#555")


    bars = ax2.bar(dates, winds, color=colors,
                   edgecolor="white", linewidth=0.7, width=0.55)

    for bar, val in zip(bars, winds):
        ax2.text(
```

```
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            f"{val}",
            ha="center", va="bottom", fontsize=8.5, color="#333",
        )


    for i, cond in enumerate(conditions):
        short = cond[:10] + "…" if len(cond) > 10 else cond
        ax2.text(i, -max(winds) * 0.18, short,
                 ha="center", va="top", fontsize=7,
                 color="#555", rotation=0)

    ax2.set_ylabel("Wind Speed (km/h)", fontsize=10)
    ax2.set_xlabel("Date", fontsize=10)
    ax2.grid(axis="y", linestyle="--", alpha=0.35)
    ax2.spines["top"].set_visible(False)
    ax2.spines["right"].set_visible(False)
    ax2.set_title("Wind Speed (km/h) + Condition", fontsize=10,
                  loc="left", color="#555")
    ax2.tick_params(axis="x", rotation=15)

    fig.tight_layout(rect=[0, 0, 1, 0.97])

    filename = os.path.join(CHARTS_DIR, f"{city_name.lower()}_dashboard.png")
    fig.savefig(filename, dpi=130, bbox_inches="tight")
    plt.close(fig)

    return filename


# Convenience: generate all three charts at once

def generate_all_charts(
    forecast: list[WeatherRecord],
    current: WeatherRecord,
```

```python
) -> tuple[str, str, str]:
    """
    Generate all three charts for a city.

    Returns:
        tuple: (temperature_path, wind_path, dashboard_path)
    """
    city = current.city
    path_temp = plot_temperature_trend(forecast, city)
    path_wind = plot_wind_speed(forecast, city)
    path_dash = plot_dashboard(forecast, current)

    return path_temp, path_wind, path_dash
```